

# AGROGEN IA

## Padrões de Engenharia Frontend

Estratégia de Dados · Axios · Offline-First · Filtros · Loading · Erros · Formulários · Performance · A11y

Documento normativo para implementação do frontend React + TypeScript + PWA

Versão 1.0 · Junho de 2026

Hackathon Expoagro Crateús 2026

# 1. Propósito e Escopo

Este documento define os padrões técnicos de engenharia para a implementação do frontend do AgroGen IA. Complementa os documentos de UX (telas, modais, fluxos) e da API (endpoints, payloads) com decisões sobre COMO o código deve ser escrito: estratégia de dados, autenticação, offline, filtros, loadings, erros, formulários, performance e acessibilidade.

## 1.1 Decisões já tomadas (não negociáveis)

- Stack: React 18 + Vite + TypeScript estrito
- Estilo: Tailwind CSS com tokens em styles.css na raiz
- Roteamento: React Router v6
- Dados: TanStack Query (React Query v5)
- HTTP: Axios com interceptors
- Formulários: React Hook Form + Zod
- UI primitives: Radix UI (Dialog, Tabs, Dropdown, Toast)
- Ícones: lucide-react
- Gráficos: Chart.js + react-chartjs-2
- PWA: vite-plugin-pwa
- Offline: estratégia média — cache de leitura + queue de mutations

## 1.2 O que NÃO está no escopo deste documento

- Testes automatizados (fora do escopo do MVP do hackathon)
- Internacionalização (pt-BR fixo)
- Storybook / design system documentado externamente
- Analytics e telemetria de uso
- CI/CD além do build padrão do Vite

# 2. Estratégia de Dados com TanStack Query

## 2.1 Configuração global do QueryClient

Um único QueryClient para a aplicação inteira, configurado em src/app/providers.tsx:

```
src/app/providers.tsx
// src/app/providers.tsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
```

```
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000, // 5 min – bom default para dados zootécnicos
      gcTime: 30 * 60 * 1000, // 30 min – cache em memória
      retry: (failureCount, error) => {
        // não tentar de novo em 4xx (erro do cliente)
        if (error.response?.status >= 400 && error.response?.status < 500) return false;
        return failureCount < 2; // 2 tentativas para 5xx e network
      },
      retryDelay: attemptIndex => Math.min(1000 * 2 ** attemptIndex, 30000),
      refetchOnWindowFocus: false, // evita refetch agressivo em PWA mobile
      refetchOnReconnect: true, // refetch ao voltar online (offline-first)
    },
    mutations: {
      retry: false, // mutations NUNCA fazem retry automático
      onError: (error) => {
        // toast genérico tratado em error-messages.ts
        showErrorToast(error);
      },
    },
  },
});
```

## 2.2 Convenção de Query Keys

Todas as query keys seguem o padrão de array hierárquico, começando pelo domínio. Permite invalidação granular ou em cascata.

| Recurso                     | Query Key                                            | Exemplo                                                       |
|-----------------------------|------------------------------------------------------|---------------------------------------------------------------|
| Lista de animais (filtrada) | ['animais', 'list', { fazendaId, filtros }]          | ['animais', 'list', { fazendaId: 'uuid', especie: 'BOVINO' }] |
| Animal individual           | ['animais', 'detail', animalId]                      | ['animais', 'detail', 'uuid-bov-0012']                        |
| Histórico do animal         | ['animais', 'detail', animalId, 'historico']         | ['animais', 'detail', 'uuid', 'historico']                    |
| Lista de inseminações       | ['inseminacoes', 'list', filtros]                    | ['inseminacoes', 'list', { resultado: 'PENDENTE' }]           |
| Diagnósticos pendentes      | ['inseminacoes', 'pendentes-diagnostico', fazendaId] | —                                                             |
| KPIs do dashboard           | ['dashboard', 'kpis', fazendaId]                     | —                                                             |
| Alertas                     | ['alertas', 'list', { lido, resolvido }]             | —                                                             |
| Contagem de alertas (sino)  | ['alertas', 'contagem']                              | —                                                             |
| Reprodutores                | ['reprodutores', 'list', filtros]                    | ['reprodutores', 'list', { especie: 'BOVINO' }]               |
| Diário — pesagens de        | ['diario', animalId, 'pesagens']                     | —                                                             |

| Recurso           | Query Key                                  | Exemplo                                  |
|-------------------|--------------------------------------------|------------------------------------------|
| animal            |                                            |                                          |
| Predição IA       | ['ia', 'predicao', animalId, contextoHash] | contextoHash = hash do input situacional |
| Usuário atual     | ['usuarios', 'me']                         | —                                        |
| Lista de fazendas | ['fazendas', 'list']                       | —                                        |

**Regra-chave**

O segundo elemento do array é SEMPRE "list" ou "detail" (ou identificador de operação como "pendentes-diagnostico", "kpis", "contagem"). Isso permite invalidações inteligentes — por exemplo, invalidar ["animais"] limpa TODAS as listas e detalhes de animais; invalidar ["animais", "list"] limpa apenas as listas mas preserva os detalhes individuais.

## 2.3 Stale Time por tipo de recurso

O default de 5 minutos cobre a maioria. Casos específicos com staleTime customizado:

| Recurso                       | staleTime                          | Justificativa                                                    |
|-------------------------------|------------------------------------|------------------------------------------------------------------|
| Lista de animais              | 5 min (default)                    | Volume médio, atualizações ocasionais                            |
| Detalhe de animal             | 5 min (default)                    | Idem                                                             |
| Inseminações (lista)          | 2 min                              | Aba "Diagnósticos pendentes" precisa estar fresca                |
| Diagnósticos pendentes        | 1 min                              | Urgência alta — usuário precisa ver o mais recente               |
| Contagem de alertas           | 60 segundos                        | Sino do header precisa atualizar com frequência                  |
| KPIs do dashboard             | 5 min (com refetchInterval 5 min)  | RNF: polling automático a cada 5 min                             |
| Lista de raças (referência)   | 24 horas                           | Dados raramente mudam                                            |
| Lista de protocolos hormonais | 24 horas                           | Idem                                                             |
| Usuário atual (me)            | 60 minutos                         | Muda raramente; refetch só ao trocar fazenda ou logout           |
| Predição IA                   | Infinity (com nova chamada manual) | Resultado é fotografia do momento; não revalidar automaticamente |

## 2.4 Padrão de Mutation e Invalidação

Toda mutation declara explicitamente o que invalida no onSuccess:

Exemplo de mutation com invalidação explícita

```
// src/lib/api/hooks/useCreateAnimal.ts
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { createAnimal } from '../endpoints/animais';

export function useCreateAnimal() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: createAnimal,
    onSuccess: (novoAnimal) => {
      // 1. Invalida todas as listas de animais (forçar refetch)
      queryClient.invalidateQueries({ queryKey: ['animais', 'list'] });

      // 2. Pre-popula o cache do detalhe para evitar fetch ao abrir SUB-01
      queryClient.setQueryData(['animais', 'detail', novoAnimal.id], novoAnimal);

      // 3. Invalida KPIs do dashboard (total de animais mudou)
      queryClient.invalidateQueries({ queryKey: ['dashboard', 'kpis'] });

      // 4. Toast de sucesso
      showSuccessToast(`Animal ${novoAnimal.codigo} cadastrado.`);
    },
  });
}
```

## 2.5 Matriz de invalidação por mutation

Tabela canônica de "ao executar mutation X, invalidar caches Y". O Claude Code DEVE seguir esta tabela:

| Mutation                            | Caches a invalidar                                                                                                                                                   | Cache a pré-popular                                 |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| POST /animais                       | ['animais','list'], ['dashboard','kpis']                                                                                                                             | ['animais','detail', {id}] ← novoAnimal             |
| PUT /animais/{id}                   | ['animais','list'], ['animais','detail',{id}]                                                                                                                        | —                                                   |
| DELETE /animais/{id}                | ['animais','list'], ['dashboard','kpis']                                                                                                                             | Remove ['animais','detail', {id}] via removeQueries |
| POST /inseminacoes                  | ['inseminacoes','list'], ['inseminacoes','pendentes-diagnostico'], ['animais','detail',{animalId}], ['dashboard','kpis'], ['alertas','list'], ['alertas','contagem'] | —                                                   |
| POST /inseminacoes/{id}/diagnostico | ['inseminacoes'], ['animais','detail',{animalId}], ['alertas']                                                                                                       | —                                                   |

| Mutation                    | Caches a invalidar                                                                                                  | Cache a pré-popular                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| POST /diario/{id}/pesagens  | ['diario',{animalId},'pesagens'], ['animais','detail',{animalId}]                                                   | —                                               |
| POST /diario/{id}/partos    | ['diario',{animalId},'partos'], ['animais','detail',{animalId}]                                                     | —                                               |
| POST /diario/{id}/sanitario | ['diario',{animalId},'sanitario'], ['alertas','contagem'] (se proxima_dose preenchida)                              | —                                               |
| POST /reprodutores          | ['reprodutores','list']                                                                                             | ['reprodutores','detail',{id}] ← novoReprodutor |
| PATCH /alertas/{id}/lido    | ['alertas','contagem']                                                                                              | Update otimista em ['alertas','list']           |
| POST /fazendas              | ['fazendas','list'], ['usuarios','me']                                                                              | —                                               |
| Troca de fazenda ativa      | ['dashboard'], ['animais'], ['inseminacoes'], ['alertas'], ['diario'], ['relatorios'] — TUDO QUE DEPENDE DE FAZENDA | —                                               |

## 2.6 Optimistic Updates — quando vale a pena

Optimistic update é quando a UI muda ANTES da confirmação do servidor. Aplicar APENAS nos casos abaixo:

| Mutation                              | Optimistic? | Por quê                                                                    |
|---------------------------------------|-------------|----------------------------------------------------------------------------|
| Marcar alerta como lido               | SIM         | Operação leve, falha rara, UX precisa ser instantânea                      |
| Marcar alerta como resolvido          | SIM         | Idem                                                                       |
| Toggle ativo/inativo de reprodutor    | SIM         | Visualização imediata do badge mudando                                     |
| Cadastro de animal                    | NÃO         | Envolve geração de código pelo servidor (BOV-XXXX); aguardar resposta real |
| Cadastro de inseminação               | NÃO         | Gera alerta no servidor; aguardar para mostrar o alerta criado             |
| Registrar pesagem                     | NÃO         | GMD calculado pelo servidor                                                |
| Registrar parto                       | NÃO         | Atualiza num_partos e data_ultimo_parto no Animal; aguardar                |
| Edições simples (nome do animal etc.) | SIM         | Operações sem efeito colateral; falha rara                                 |
| Predição IA                           | NÃO         | O resultado é o ponto da operação                                          |

## Padrão de optimistic update

```

Padrão de optimistic update
// src/lib/api/hooks/useMarcarAlertaLido.ts
export function useMarcarAlertaLido() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: ({ alertaId }: { alertaId: string }) =>
      api.patch(`/alertas/${alertaId}/lido`, { lido: true }),

    onMutate: async ({ alertaId }) => {
      // 1. Cancela queries em andamento que poderiam sobrescrever
      await queryClient.cancelQueries({ queryKey: ['alertas'] });

      // 2. Snapshot do estado anterior (para rollback)
      const previous = queryClient.getQueryData(['alertas', 'contagem']);

      // 3. Update otimista
      queryClient.setQueryData(['alertas', 'contagem'], (old: any) => ({
        ...old,
        nao_lidos: Math.max(0, (old?.nao_lidos ?? 0) - 1),
      }));

      return { previous };
    },

    onError: (err, vars, context) => {
      // Rollback em caso de falha
      if (context?.previous) {
        queryClient.setQueryData(['alertas', 'contagem'], context.previous);
      }
    },

    onSettled: () => {
      // Sempre revalidar ao final, sucesso ou falha
      queryClient.invalidateQueries({ queryKey: ['alertas'] });
    },
  });
}

```

## 3. Camada Axios e Autenticação

### 3.1 Instância Axios única

Uma única instância configurada e exportada de `src/lib/api/client.ts`. Todos os endpoints consomem esta instância — nada de chamar axios diretamente em outros arquivos.

```

src/lib/api/client.ts
// src/lib/api/client.ts
import axios from 'axios';

export const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL ?? '/api/v1',

```

```

    timeout: 15000, // 15s – limite generoso para operações pesadas
    (PDF)
    headers: { 'Content-Type': 'application/json' },
  });

```

## 3.2 Interceptor de Request — injeção automática

Antes de cada request, injetar automaticamente:

- Authorization: Bearer {access\_token} do localStorage
- X-Fazenda-ID: UUID da fazenda ativa (de useFazendaAtiva)
- AbortController para permitir cancelamento (TanStack Query usa isso automaticamente em useQuery)

### Interceptor de Request

```

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('agrogen.accessToken');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  const fazendaAtivaId = localStorage.getItem('agrogen.fazendaAtivaId');
  if (fazendaAtivaId && !config.url?.startsWith('/auth/')) {
    config.headers['X-Fazenda-ID'] = fazendaAtivaId;
  }

  return config;
});

```

## 3.3 Interceptor de Response — tratamento global de erros

Pós-processa toda resposta para detectar token expirado, 401, 403, 500, network.

### Interceptor de Response com refresh token

```

let refreshPromise: Promise<string> | null = null;

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const original = error.config;
    const status = error.response?.status;

    // 401 com token expirado: tenta refresh UMA VEZ
    if (status === 401 && !original._retry && !original.url?.includes('/auth/refresh')) {
      original._retry = true;

      try {
        // Deduplicação: se já há refresh em andamento, espera
        if (!refreshPromise) {
          refreshPromise = refreshAccessToken();
        }
        const novoToken = await refreshPromise;
        refreshPromise = null;
      } catch {
        // Se o refresh falhar, não fazemos nada aqui
      }
    }

    return Promise.reject(error);
  }
);

```



```

    // Retenta a request original com novo token
    original.headers.Authorization = `Bearer ${novoToken}`;
    return api(original);
  } catch (refreshError) {
    refreshPromise = null;
    // Refresh falhou: logout silencioso e redirect para login
    localStorage.removeItem('agrogen.accessToken');
    localStorage.removeItem('agrogen.refreshToken');
    window.location.href = '/login?session=expired';
    return Promise.reject(refreshError);
  }
}

// 403: não tem permissão – toast e segue
if (status === 403) {
  showErrorToast('Você não tem permissão para esta ação.');
```

```

}

// Network error (offline): tratado pelo módulo offline (seção 4)
if (!error.response) {
  handleNetworkError(error);
}

return Promise.reject(error);
}
);
```

### 3.4 Deduplicação de Refresh Token

#### Por que isso é crítico

Quando o token expira, várias queries do React Query disparam 401 ao mesmo tempo (dashboard, lista de animais, alertas). Sem deduplicação, cada uma tentaria fazer refresh paralelamente, invalidando os refresh tokens uns dos outros (rotação) e causando logout em cadeia. A variável `refreshPromise` garante que apenas UMA chamada de refresh acontece por vez; todas as outras aguardam o mesmo resultado.

### 3.5 Hook `useAuth`

Centraliza estado de autenticação. Implementação enxuta em `src/hooks/useAuth.ts`:

```

src/hooks/useAuth.ts
export function useAuth() {
  const queryClient = useQueryClient();
  const { data: usuario } = useQuery({
    queryKey: ['usuarios', 'me'],
    queryFn: getUsuarioAtual,
    staleTime: 60 * 60 * 1000, // 1h
    enabled: !!localStorage.getItem('agrogen.accessToken'),
  });

  const login = async (email: string, senha: string, lembrar: boolean) => {
    const resp = await api.post('/auth/login', { email, senha });
    localStorage.setItem('agrogen.accessToken', resp.data.data.access_token);
    if (lembrar) {
```

```

    localStorage.setItem('agrogen.refreshToken', resp.data.data.refresh_token);
  } else {
    sessionStorage.setItem('agrogen.refreshToken', resp.data.data.refresh_token);
  }
  queryClient.invalidateQueries({ queryKey: ['usuarios', 'me'] });
};

const logout = async () => {
  const refresh = localStorage.getItem('agrogen.refreshToken')
    ?? sessionStorage.getItem('agrogen.refreshToken');
  if (refresh) await api.post('/auth/logout', { refresh_token:
refresh }).catch(()=>{});
  localStorage.clear();
  sessionStorage.clear();
  queryClient.clear();
  window.location.href = '/login';
};

return { usuario, login, logout, isAuthenticated: !!usuario };
}

```

### 3.6 Cancelamento de requests

TanStack Query passa um `AbortSignal` automaticamente. Cada endpoint deve aceitar e repassar ao Axios:

```

Cancelamento via AbortSignal
// src/lib/api/endpoints/animais.ts
export async function listarAnimais(
  filtros: AnimalFilters,
  signal?: AbortSignal,
): Promise<Paginated<Animal>> {
  const { data } = await api.get('/animais', { params: filtros, signal });
  return data;
}

// Uso no useQuery (signal é injetado automaticamente):
useQuery({
  queryKey: ['animais', 'list', filtros],
  queryFn: ({ signal }) => listarAnimais(filtros, signal),
});

```

Sem isso, requests pendentes continuam após o usuário sair da tela, gastando largura de banda e bateria.

## 4. Estratégia Offline-First (Nível Médio)

O AgroGen IA opera em condições reais do sertão cearense — conexão móvel intermitente, fazendas em áreas com cobertura fraca. A estratégia offline de nível médio adotada permite (a) leitura completa do cache quando offline e (b) enfileirar mutations para sincronizar quando a conexão voltar.

## 4.1 O que funciona OFFLINE

| Funcionalidade                            | Comportamento offline                                          | Como                                                |
|-------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------|
| Leitura de dados já carregados            | Funciona normalmente (cache do React Query + service worker)   | staleTime alto + cache persistido                   |
| Listagem de animais visitada recentemente | Lê do cache, mostra banner "Dados de {data} — sem atualização" | Service worker NetworkFirst com fallback CacheFirst |
| Cadastro de animal                        | Permite preencher, fica em queue local                         | Mutation enfileirada                                |
| Registrar paisagem                        | Permite preencher, fica em queue local                         | Mutation enfileirada                                |
| Registrar inseminação                     | Permite preencher, fica em queue local                         | Mutation enfileirada                                |
| Registrar evento sanitário                | Permite preencher, fica em queue local                         | Mutation enfileirada                                |
| Registrar ocorrência                      | Permite preencher, fica em queue local                         | Mutation enfileirada                                |
| Marcar alerta lido/resolvido              | Optimistic — fica em queue                                     | Mutation enfileirada                                |

## 4.2 O que NÃO funciona offline

- Login/cadastro/recuperação de senha (requer rede para autenticar)
- Predição IA (precisa chamar o modelo no servidor)
- Padrões de Fertilidade (cálculos pesados no servidor)
- Seleção Genética (idem)
- Geração de PDF (Diário, Relatórios — backend)
- Importação CSV (requer upload)
- Exclusão de qualquer entidade (decisão de produto — operação destrutiva exige confirmação online)

## 4.3 Detecção de online/offline

Hook útil consumido por toda a aplicação:

```
src/hooks/useOnlineStatus.ts
// src/hooks/useOnlineStatus.ts
import { useEffect, useState } from 'react';

export function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);

  useEffect(() => {
    const goOnline = () => setIsOnline(true);
    const goOffline = () => setIsOnline(false);
    window.addEventListener('online', goOnline);
    window.addEventListener('offline', goOffline);
    return () => {
```

```

        window.removeEventListener('online', goOnline);
        window.removeEventListener('offline', goOffline);
    };
}, []);

return isOnline;
}

```

## 4.4 Banner global de status offline

Componente fixo no topo, abaixo do header, visível em todas as telas autenticadas:

| Estado                  | Visual                                                                                     | Comportamento                                                                                   |
|-------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Online (default)        | Sem banner                                                                                 | UI normal                                                                                       |
| Offline detectado       | Banner amarelo: "Você está offline. Mudanças serão sincronizadas quando a conexão voltar." | Persistente enquanto offline. Mostra contador de ações pendentes se houver: "(3 ações na fila)" |
| Voltou online com queue | Banner azul: "Conexão restaurada. Sincronizando 3 ações..."                                | Mostra spinner. Persiste até a queue zerar.                                                     |
| Sincronização concluída | Toast verde temporário: "Sincronização concluída. 3 ações enviadas."                       | 4 segundos e some.                                                                              |
| Falha na sincronização  | Banner vermelho: "Não conseguimos sincronizar X ações. [Ver detalhes]"                     | Link abre drawer com lista das ações que falharam e botão "Tentar de novo"                      |

## 4.5 Mutation Queue — implementação

A queue é persistida em IndexedDB (não localStorage — IndexedDB suporta volume maior e operações assíncronas). Cada item da queue contém metadados suficientes para reexecutar a mutation quando online.

### Estrutura do item da queue

```

Tipo do item da queue
interface QueuedMutation {
  id: string; // UUID local
  timestamp: number; // criação no client
  endpoint: string; // ex: '/animais', '/inseminacoes'
  method: 'POST' | 'PATCH' | 'PUT';
  payload: unknown; // body da request
  invalidateKeys: QueryKey[]; // o que invalidar após sucesso
  optimisticData?: unknown; // dados otimistas mostrados na UI
  retries: number; // tentativas até agora
  lastError?: string; // mensagem da última falha (para debug)
  status: 'pending' | 'syncing' | 'failed';
}

```

## Biblioteca recomendada

Usar idb-keyval (~600 bytes) para wrapper simples sobre IndexedDB. Evitar dependências pesadas como Dexie no MVP. Implementar a queue como pequeno módulo em `src/lib/offline/mutationQueue.ts`:

```
src/lib/offline/mutationQueue.ts
// src/lib/offline/mutationQueue.ts
import { get, set, del, keys } from 'idb-keyval';

const STORE_PREFIX = 'queue:';

export async function enqueue(mutation: Omit<QueuedMutation,
'id'|'timestamp'|'retries'|'status'>) {
  const item: QueuedMutation = {
    ...mutation,
    id: crypto.randomUUID(),
    timestamp: Date.now(),
    retries: 0,
    status: 'pending',
  };
  await set(`${STORE_PREFIX}${item.id}`, item);
  return item.id;
}

export async function listQueue(): Promise<QueuedMutation[]> {
  const allKeys = await keys();
  const queueKeys = (allKeys as string[]).filter(k => k.startsWith(STORE_PREFIX));
  return Promise.all(queueKeys.map(k => get(k))) as Promise<QueuedMutation[]>;
}

export async function removeFromQueue(id: string) {
  await del(`${STORE_PREFIX}${id}`);
}

export async function markFailed(id: string, error: string) {
  const item = await get<QueuedMutation>(`${STORE_PREFIX}${id}`);
  if (item) {
    item.retries += 1;
    item.lastError = error;
    item.status = 'failed';
    await set(`${STORE_PREFIX}${id}`, item);
  }
}
```

## 4.6 Processador da queue (sync engine)

Hook que escuta mudança de status para online e dispara o processamento sequencial da queue. Sequencial (não paralelo) para garantir ordem das operações — ex: criar Animal antes de criar Pesagem que depende dele.

```
src/lib/offline/useQueueSync.ts
// src/lib/offline/useQueueSync.ts
export function useQueueSync() {
  const isOnline = useOnlineStatus();
```

```

const queryClient = useQueryClient();
const [syncing, setSyncing] = useState(false);

useEffect(() => {
  if (!isOnline || syncing) return;
  processQueue();

  async function processQueue() {
    setSyncing(true);
    const items = await listQueue();
    const pending = items.filter(i => i.status !== 'syncing');

    for (const item of pending) {
      try {
        await api.request({
          url: item.endpoint,
          method: item.method,
          data: item.payload,
        });
        await removeFromQueue(item.id);
        item.invalidateKeys.forEach(key =>
          queryClient.invalidateQueries({ queryKey: key })
        );
      } catch (err) {
        await markFailed(item.id, String(err));
        if (err.response?.status >= 400 && err.response?.status < 500) {
          // 4xx: erro definitivo, não tenta de novo. Notifica usuário.
          showFailedSyncToast(item, err);
        }
        // Para outras falhas, próxima tentativa quando voltar online de novo
      }
    }
    setSyncing(false);
  }
}, [isOnline]);

return { syncing };
}

```

## 4.7 Mutations que respeitam a queue

Cada mutation deve verificar se está offline antes de chamar a API:

```

Mutation com fallback offline
// Padrão para mutations offline-aware
export function useCreateAnimalOfflineAware() {
  const queryClient = useQueryClient();
  const isOnline = useOnlineStatus();

  return useMutation({
    mutationFn: async (input: CreateAnimalInput) => {
      if (!isOnline) {
        // Enfileira para depois
        const queueId = await enqueue({
          endpoint: '/animais',
          method: 'POST',
          payload: input,
          invalidateKeys: [['animais', 'list'], ['dashboard', 'kpis']],
        });
        // Atualiza UI otimisticamente com dados temporários
      }
    }
  });
}

```

```

    return { id: `local:${queueId}`, ...input, codigo: 'PENDENTE' };
  }
  // Online: chamada normal
  const { data } = await api.post('/animais', input);
  return data.data;
},
onSuccess: (animal) => {
  queryClient.invalidateQueries({ queryKey: ['animais', 'list'] });
  if (animal.id.startsWith('local:')) {
    showInfoToast('Animal salvo localmente. Será enviado quando a conexão voltar.');
```

## 4.8 Indicadores visuais de dados pendentes

- Animais criados offline aparecem na lista com badge cinza "PENDENTE" no lugar do código
- Inseminações offline aparecem com ícone de relógio cinza ao lado da data
- Contador no banner offline: "(3 ações na fila)"
- Ao clicar no contador, abre drawer com lista das ações pendentes — usuário pode revisar ou cancelar

## 4.9 Limite de queue

### Política de limite

A queue tem limite de 50 mutations pendentes. Ao atingir, novas mutations são bloqueadas com mensagem: "Há muitas ações aguardando sincronização. Conecte-se à internet para enviar antes de adicionar mais." Isso evita o usuário acumular semanas de trabalho offline e descobrir um desastre ao sincronizar.

# 5. Filtros e Busca

## 5.1 Princípios gerais

- Filtros do tipo "chip/pill" para enumerações simples (Espécie, Sexo, Status) — visíveis e clicáveis na primeira linha
- Filtros avançados (range, multi-select, datas específicas) vão para drawer lateral (DRAWER-02, DRAWER-03)
- Combinação dos filtros é sempre AND lógico (interseção)
- Multi-select dentro de UM filtro é OR lógico (ex: Bovino OR Ovino)
- Filtros persistem em sessionStorage durante a sessão; resetam ao recarregar
- Botão "Limpar filtros" aparece apenas quando ao menos um filtro está ativo

- Cada chip ativo exibe contagem de resultados ao lado: "Bovino (42)"

## 5.2 Padrão de tipos de filtro

| Tipo de filtro         | Componente UI               | Quando usar                              | Exemplos                                                    |
|------------------------|-----------------------------|------------------------------------------|-------------------------------------------------------------|
| Chip único             | Pill button toggle          | Enumeração simples com 2-5 opções, comum | Sexo (M/F), Status do resultado, Tipo de IA                 |
| Chip multi-select      | Grupo de pills com toggle   | Categorias combináveis                   | Espécies (pode ter Bovino+Ovino), Tipos de evento sanitário |
| Date range com atalhos | Inputs de data + presets    | Filtros temporais — sempre com atalhos   | Período em Inseminação, Relatórios                          |
| Range numérico         | Slider duplo OU dois inputs | Faixas contínuas                         | Faixa de peso, faixa de idade                               |
| Autocomplete           | Combobox com busca          | Lista grande de opções textuais          | Raça (50+ opções), Técnico responsável                      |
| Busca textual livre    | Input com debounce 300ms    | Busca por nome, código, brinco           | Campo "q" da listagem de animais                            |
| Toggle binário         | Switch                      | Estado on/off (incluir/excluir)          | Apenas ativos, Apenas não lidos                             |

## 5.3 Range numérico — recomendação por contexto

| Contexto                   | Recomendação                                                | Justificativa                                                           |
|----------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------|
| Faixa de peso (animal)     | Dois inputs (min/max) com label "kg"                        | Mobile-first: sliders são imprecisos no toque. Input numérico é direto. |
| Faixa de idade             | Dois inputs (min/max) com unidade selecionável (meses/anos) | Usuários pensam em meses para jovens e anos para adultos                |
| Faixa de condição corporal | Slider duplo (1 a 5)                                        | Range pequeno e visual                                                  |
| Número de partos           | Dois inputs (min/max) integer                               | Valores discretos                                                       |
| DEP fertilidade            | Dois inputs (min/max) decimal                               | Valores técnicos precisos                                               |

## 5.4 Date range — atalhos obrigatórios

Todo seletor de período inclui chips de atalho ANTES dos inputs manuais:

- Hoje
- Últimos 7 dias
- Este mês



- Último trimestre
- Este ano
- Tudo (limpa range)

Clicar em um atalho preenche os inputs e dispara o filtro. Inputs manuais ficam abaixo para casos específicos.

## 5.5 Debounce de busca textual

Toda busca em tempo real usa debounce de 300ms. Hook reutilizável:

```
useDebounce + uso em query
// src/hooks/useDebounce.ts
export function useDebounce<T>(value: T, delay = 300): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

// Uso na TELA-02:
const [busca, setBusca] = useState('');
const buscaDebounce = useDebounce(busca, 300);
const { data } = useQuery({
  queryKey: ['animais', 'list', { q: buscaDebounce, ...filtros }],
  queryFn: ({ signal }) => listarAnimais({ q: buscaDebounce, ...filtros }, signal),
  enabled: buscaDebounce.length === 0 || buscaDebounce.length >= 2,
});
```

## 5.6 Reset granular vs total

- Cada chip ativo tem "x" para remover individualmente
- Botão "Limpar filtros" no topo limpa TODOS de uma vez
- Em drawers (DRAWER-02, DRAWER-03), botão "Limpar tudo" reseta o drawer; botão "Aplicar" fecha e aplica
- Filtros que NÃO foram explicitamente alterados pelo usuário não aparecem como chips ativos

## 5.7 Detalhamento dos drawers de filtros avançados

### DRAWER-02 — Filtros Avançados de Animais

| Filtro          | Tipo                      | Comportamento                                                                |
|-----------------|---------------------------|------------------------------------------------------------------------------|
| Raça específica | Multi-select autocomplete | Lista carregada dinamicamente conforme espécie já filtrada nos chips rápidos |

| Filtro                     | Tipo                                      | Comportamento                                          |
|----------------------------|-------------------------------------------|--------------------------------------------------------|
| Faixa de peso              | Dois inputs (min/max kg)                  | Validação por espécie                                  |
| Faixa de idade             | Dois inputs (min/max) + select meses/anos | —                                                      |
| Faixa de número de partos  | Dois inputs (min/max integer)             | Para fêmeas — caso macho selecionado nos chips, oculto |
| Data de nascimento (range) | Date range com atalhos                    | Útil para safras específicas                           |
| Condição corporal (range)  | Slider duplo 1-5                          | —                                                      |
| Brinco (busca)             | Input texto                               | Busca exata                                            |
| Fazenda                    | Select                                    | Aparece somente se usuário tem múltiplas fazendas      |

### DRAWER-03 — Filtros Avançados de Inseminações

| Filtro                   | Tipo                                     | Comportamento                          |
|--------------------------|------------------------------------------|----------------------------------------|
| Animal específico        | Autocomplete por código ou nome          | Útil para histórico individual         |
| Reprodutor específico    | Autocomplete por nome                    | —                                      |
| Tipo de IA               | Multi-select chips (IA Conv / IATF / TE) | —                                      |
| Protocolo hormonal       | Multi-select autocomplete                | Lista vinda da TELA-11                 |
| Faixa de CC no momento   | Slider duplo 1-5                         | —                                      |
| Resultado do diagnóstico | Multi-select chips                       | Pendente / Prenha / Vazia              |
| Técnico responsável      | Multi-select autocomplete                | Apenas usuários com perfil Técnico/Vet |
| Período                  | Date range com atalhos                   | —                                      |

## 6. Loading, Skeleton e Estados Intermediários

### 6.1 Hierarquia de estados

Toda query/mutation passa por estados específicos. Definir comportamento em cada um:

| Estado            | Indicador visual                       | Quando usar                                         |
|-------------------|----------------------------------------|-----------------------------------------------------|
| idle              | Nenhum                                 | Antes da primeira execução (useQuery enabled:false) |
| pending (inicial) | Skeleton screen                        | Primeira chamada, ainda sem dados em cache          |
| pending (refetch) | Manter dados antigos + indicador sutil | Refetch em background, dados em cache existem       |
| success           | Conteúdo normal                        | —                                                   |
| error             | Estado de erro (banner ou tela cheia)  | Conforme seção 7                                    |
| mutation pending  | Botão com spinner + disabled           | Submit de formulário                                |
| empty             | Empty state da tela (do Adendo UX)     | Query bem-sucedida sem dados                        |

## 6.2 Catálogo de Skeletons

Cada tela tem um skeleton específico. Componentes em src/components/ui/skeletons/:

| Componente                     | Onde usar                                             | Estrutura                                                                                |
|--------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------|
| <TableSkeleton rows={n} />     | TELA-02, TELA-03, TELA-11, qualquer tabela            | Cabeçalho + n linhas de blocos cinzas com larguras variadas. Padrão: rows=10             |
| <CardListSkeleton count={n} /> | Mobile (cards empilhados)                             | n cards com placeholder de título, subtítulo e 3 metadados                               |
| <KPICardSkeleton />            | Dashboard                                             | Bloco do KPI: 1 linha (título) + 1 número grande + 1 subtítulo. Renderizar 4 lado a lado |
| <ChartSkeleton />              | Dashboard, Padrões IA, Relatórios                     | Retângulo com altura fixa e ícone "chart" cinza centralizado                             |
| <AnimalProfileSkeleton />      | SUB-01 Perfil do Animal                               | Header com avatar + 3 linhas + 3 cards de bloco                                          |
| <FormSkeleton fields={n} />    | Modais que carregam dados antes de exibir (modo edit) | n linhas de label + input cinza                                                          |
| <DiarioTabSkeleton />          | Abas do Diário de Bordo                               | Gráfico + tabela acima                                                                   |

## 6.3 Comportamento durante refetch

### Regra principal

NUNCA substituir conteúdo já carregado por skeleton durante refetch. O usuário deve continuar

vendo os dados atuais. Use `placeholderData: keepPreviousData` ou indicador sutil (ex: pequeno spinner ao lado do título da seção) para sinalizar que está atualizando.

#### Padrão de refetch com `keepPreviousData`

```
// Exemplo: lista de animais com paginação
const { data, isPlaceholderData, isFetching } = useQuery({
  queryKey: ['animais', 'list', filtros],
  queryFn: ({signal}) => listarAnimais(filtros, signal),
  placeholderData: (prev) => prev, // mantém dados antigos durante refetch
});

// Na UI:
<div className="flex items-center gap-2">
  <h2>Animais</h2>
  {isFetching && <SpinnerSm aria-label="Atualizando" />}
</div>
{data.length === 0 && !isFetching ? <EmptyState /> : <Table rows={data} />}
```

## 6.4 Loading em botões de submit

Todo botão de submit em formulário ou ação destrutiva deve:

- Mostrar spinner inline (lucide Loader2 com `animate-spin`) à esquerda do texto
- Manter o texto original do botão (NÃO trocar para "Carregando...") — evita layout shift
- Ficar disabled durante a operação
- Voltar ao estado normal apenas após a Promise resolver (sucesso ou falha)

#### Botão com loading inline

```
// src/components/ui/Button.tsx
<Button onClick={salvar} disabled={mutation.isPending}>
  {mutation.isPending && <Loader2 className="animate-spin mr-2 h-4 w-4" />}
  Salvar Animal
</Button>
```

## 6.5 Loading da Predição IA — caso especial

A Predição IA tem latência alta (até 1s) e impacto na UX. Implementar animação com mensagens rotativas para parecer "trabalhando" mesmo quando rápido:

| Tempo decorrido | Mensagem exibida                                                 |
|-----------------|------------------------------------------------------------------|
| 0 - 500 ms      | "Lendo o histórico do animal..."                                 |
| 500 ms - 1 s    | "Cruzando com dados genéticos..."                                |
| 1 s - 2 s       | "Calculando a probabilidade..."                                  |
| > 2 s           | "Quase pronto..."                                                |
| > 5 s           | Adicionar opção "Cancelar" e mensagem "Está demorando mais que o |

| Tempo decorrido | Mensagem exibida |
|-----------------|------------------|
|                 | esperado..."     |

Mensagens são puramente decorativas — não refletem o que acontece no backend. O propósito é dar feedback de progresso.

## 6.6 Loading da geração de PDF

Geração de PDF (Diário, Relatórios) pode levar 3-10 segundos. Padrão de UX:

- Botão fica em estado de loading com spinner
- Após 3 segundos sem resposta: exibir banner inline acima do botão: "Geração em andamento. Continue navegando — avisaremos quando estiver pronto."
- Quando o backend responde com o arquivo, dispara download automático e toast "PDF gerado!"
- Permitir que o usuário navegue para outras telas sem cancelar a geração

## 7. Tratamento de Erros — Camada de Código

O Adendo UX seção 7 define os canais visuais (toast/banner/modal/tela cheia). Esta seção complementa com a implementação no código.

### 7.1 Arquivo central de mensagens

Único arquivo que mapeia códigos da API para mensagens em pt-BR:

```
src/lib/api/error-messages.ts
// src/lib/api/error-messages.ts
type ErrorChannel = 'toast' | 'banner' | 'inline' | 'modal' | 'fullscreen';

interface ErrorMapping {
  message: string;
  channel: ErrorChannel;
  duration?: number; // ms — só para toast
}

export const ERROR_MESSAGES: Record<string, ErrorMapping> = {
  AUTH_INVALID_CREDENTIALS: { message: 'E-mail ou senha incorretos. Tente novamente.',
    channel: 'banner' },
  AUTH_ACCOUNT_LOCKED: { message: 'Sua conta foi temporariamente bloqueada. Tente
    em alguns minutos.', channel: 'banner' },
  TOKEN_EXPIRED: { message: 'Sua sessão expirou.', channel: 'toast' },
  EMAIL_ALREADY_EXISTS: { message: 'Este e-mail já está cadastrado.', channel:
    'inline' },
  CPF_ALREADY_EXISTS: { message: 'Este CPF já está vinculado a outra conta.',
    channel: 'inline' },
  VALIDATION_ERROR: { message: 'Há campos com erro.', channel: 'inline' },
  ANIMAL_NOT_FOUND: { message: 'Animal não encontrado.', channel: }
```

```

'fullscreen' },
  INVALID_PESO: { message: 'Peso fora da faixa válida para a espécie.',
channel: 'inline' },
  INVALID_DATE: { message: 'Data inválida.', channel: 'inline' },
  BRINCO_DUPLICADO: { message: 'Este brinco já está cadastrado em outro
animal.', channel: 'inline' },
  INTERVALO_CURTO: { message: 'Atenção: intervalo curto desde a última
inseminação.', channel: 'banner' },
  STATUS_INCOMPATIVEL: { message: 'Status do animal incompatível.', channel:
'banner' },
  DIAGNOSTICO_JA_EXISTE: { message: 'Esta inseminação já tem diagnóstico.', channel:
'toast' },
  HISTORICO_INSUFICIENTE: { message: 'IA precisa de pelo menos uma inseminação
anterior.', channel: 'banner' },
  IA_SERVICE_UNAVAILABLE: { message: 'Análise por IA temporariamente indisponível.',
channel: 'banner' },
  DADOS_INSUFICIENTES: { message: 'Dados insuficientes para esta análise.',
channel: 'inline' },
  FORBIDDEN: { message: 'Seu perfil não tem permissão para esta ação.',
channel: 'toast' },
  ANIMAL_MACHO: { message: 'Aba disponível apenas para fêmeas.', channel:
'inline' },
  REPRODUTOR_TEM_VINCULOS: { message: 'Reprodutor com inseminações não pode ser
excluído.', channel: 'modal' },
  NETWORK_ERROR: { message: 'Sem conexão. Verifique sua internet.', channel:
'banner' },
  INTERNAL_SERVER_ERROR: { message: 'Algo deu errado do nosso lado. Tente
novamente.', channel: 'toast' },
  DEFAULT: { message: 'Ocorreu um erro. Tente novamente.', channel:
'toast' },
};

export function getErrorMessage(error: unknown): ErrorMapping {
  const codigo = (error as any)?.response?.data?.error?.codigo;
  return ERROR_MESSAGES[codigo] ?? ERROR_MESSAGES.DEFAULT;
}

```

## 7.2 ErrorBoundary global

Captura erros JavaScript não tratados (crash de render). Substitui a árvore de componentes por tela de erro amigável e oferece "Recarregar".

```

ErrorBoundary
// src/components/ErrorBoundary.tsx
import { Component, type ReactNode } from 'react';

interface State { hasError: boolean; error?: Error; }

export class ErrorBoundary extends Component<{children:ReactNode}, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }

  componentDidCatch(error: Error, info: React.ErrorInfo) {
    console.error('Unhandled error:', error, info);
    // Aqui poderia enviar para Sentry, Datadog etc. (fora do MVP)
  }
}

```

```

render() {
  if (this.state.hasError) {
    return <FullScreenError onReload={() => window.location.reload()} />;
  }
  return this.props.children;
}
}

// Uso em providers.tsx envolvendo a aplicação inteira

```

## 7.3 Erro 404 — Rota inexistente

Catch-all no React Router exibe tela amigável:

```

Rota 404
// src/app/router.tsx
{
  path: '*',
  element: <NotFoundPage />,
}

// NotFoundPage.tsx — conteúdo:
// - Ícone (lucide map-pin-off)
// - Título: 'Página não encontrada'
// - Descrição: 'O endereço que você tentou acessar não existe.'
// - CTA: 'Voltar para o início' (Link to=/dashboard)

```

## 7.4 Erro 500 / network em queries

Padrão de tela de erro inline em useQuery:

```

Padrão de renderização condicional
const { data, error, isPending, refetch } = useQuery({...});

if (isPending) return <TableSkeleton rows={10} />;

if (error) {
  return (
    <ErrorState
      message={getErrorMessage(error).message}
      onRetry={refetch}
    />
  );
}

if (!data?.length) return <EmptyState ... />;

return <Table rows={data} />;

```

## 8. Validação de Formulários (React Hook Form + Zod)

### 8.1 Princípios

- Toda lógica de validação fica em schemas Zod — fonte única de verdade
- O mesmo schema serve para validação no client E geração de tipos TypeScript
- Schemas reutilizados entre create e edit (com refinements quando necessário)
- Validação ao perder foco (onBlur) por padrão; revalidação ao digitar após o primeiro erro
- Mensagens de erro sempre em pt-BR, inline abaixo do campo
- Submit desabilitado até o formulário estar válido
- Validação assíncrona (CPF único, brinco único) com debounce de 500ms

### 8.2 Estrutura de schemas

Organização em src/lib/schemas/ — um arquivo por entidade:

#### Estrutura de schemas

```
src/lib/schemas/
  animal.ts           // schemas de Animal
  inseminacao.ts      // schemas de Inseminação
  reprodutor.ts       // schemas de Reprodutor
  pesagem.ts
  parto.ts
  sanitario.ts
  ocorrencia.ts
  fazenda.ts
  auth.ts             // login, cadastro, recuperação
  usuario.ts          // perfil
  index.ts            // re-export central
```

### 8.3 Padrão de schema com reuso create/edit

```
src/lib/schemas/animal.ts
// src/lib/schemas/animal.ts
import { z } from 'zod';

// Schema BASE com todos os campos comuns
const animalBaseSchema = z.object({
  nome: z.string().min(2, 'Mínimo 2 caracteres').max(100),
  sexo: z.enum(['MACHO', 'FEMEA']),
  data_nascimento: z.string().refine(
    (v) => new Date(v) <= new Date(),
    'Data de nascimento não pode ser futura'
  ),
  peso_inicial_kg: z.number().positive('Peso deve ser positivo'),
  condicao_corporal: z.number().int().min(1).max(5),
  fazenda_id: z.string().uuid(),
  brinco: z.string().optional(),
});
```



```
// Schema CREATE: espécie é editável e obrigatória
export const createAnimalSchema = animalBaseSchema.extend({
  especie: z.enum(['BOVINO', 'OVINO', 'CAPRINO']),
}).superRefine((data, ctx) => {
  // Validação por espécie
  const faixas = {
    BOVINO: { min: 50, max: 900 },
    OVINO: { min: 10, max: 120 },
    CAPRINO: { min: 8, max: 100 },
  };
  const f = faixas[data.especie];
  if (data.peso_inicial_kg < f.min || data.peso_inicial_kg > f.max) {
    ctx.addIssue({
      path: ['peso_inicial_kg'],
      code: 'custom',
      message: `Peso para ${data.especie} deve estar entre ${f.min} e ${f.max} kg`,
    });
  }
});

// Schema EDIT: espécie é read-only (não validamos no submit, é imutável)
export const editAnimalSchema = animalBaseSchema;

// Tipos derivados automaticamente
export type CreateAnimalInput = z.infer<typeof createAnimalSchema>;
export type EditAnimalInput = z.infer<typeof editAnimalSchema>;
```

## 8.4 Integração com React Hook Form

### AnimalModal.tsx — uso típico

```
// src/components/modals/AnimalModal.tsx
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { createAnimalSchema, type CreateAnimalInput } from '@lib/schemas/animal';

interface AnimalModalProps {
  mode: 'create' | 'edit';
  data?: Animal;
  onClose: () => void;
}

export function AnimalModal({ mode, data, onClose }: AnimalModalProps) {
  const schema = mode === 'create' ? createAnimalSchema : editAnimalSchema;

  const {
    register, handleSubmit, watch, formState: { errors, isValid },
  } = useForm<CreateAnimalInput>({
    resolver: zodResolver(schema),
    defaultValues: mode === 'edit' ? data : { sexo: 'FEMEA', condicao_corporal: 3 },
    mode: 'onBlur',
    reValidateMode: 'onChange',
  });

  const createMutation = useCreateAnimal();
  const updateMutation = useUpdateAnimal();

  const onSubmit = (input: CreateAnimalInput) => {
    if (mode === 'create') createMutation.mutate(input, { onSuccess: onClose });
    else updateMutation.mutate({ id: data!.id, input }, { onSuccess: onClose });
  };
}
```

```

const especieAtual = watch('especie');
// Pode usar especieAtual para carregar lista dinâmica de raças, etc.

return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <FormField label="Nome" error={errors.nome?.message}>
      <Input {...register('nome')} />
    </FormField>
    { /* ... demais campos ... */ }
    <Button type="submit" disabled={!isValid || createMutation.isPending}>
      {createMutation.isPending && <Loader2 className="animate-spin mr-2 h-4 w-4" />}
      {mode === 'create' ? 'Salvar Animal' : 'Salvar Alterações'}
    </Button>
  </form>
);
}

```

## 8.5 Validação assíncrona (CPF e Brinco únicos)

Alguns campos precisam validar no servidor antes do submit (CPF não pode estar duplicado, brinco deve ser único na fazenda). Implementação com debounce de 500ms:

```

Validação assíncrona com debounce
// src/hooks/useAsyncValidation.ts
export function useAsyncFieldValidation(
  fieldName: string,
  value: string,
  validateFn: (v: string) => Promise<{ valid: boolean; message?: string }>,
) {
  const debounced = useDebounce(value, 500);
  const { data, isPending } = useQuery({
    queryKey: ['async-validation', fieldName, debounced],
    queryFn: () => validateFn(debounced),
    enabled: debounced.length >= 3,
    staleTime: 60 * 1000,
  });
  return { isValidating: isPending, result: data };
}

// Uso no modal de cadastro de Animal – valida brinco:
const brinco = watch('brinco');
const { isValidating, result } = useAsyncFieldValidation(
  'brinco',
  brinco ?? '',
  async (v) => {
    const resp = await api.get('/animais', { params: { brinco: v, exact: true } });
    return { valid: resp.data.data.length === 0, message: 'Brinco já cadastrado' };
  }
);

// Indicar no UI:
<FormField
  label="Brinco"
  hint={isValidating ? 'Verificando...' : undefined}
  error={result?.valid === false ? result.message : undefined}
>
  <Input {...register('brinco')} />
</FormField>

```

## 8.6 Componente FormField padrão

Wrapper único que padroniza label, input, hint, error e aria-attributes:

```
FormField.tsx
// src/components/ui/FormField.tsx
interface FormFieldProps {
  label: string;
  error?: string;
  hint?: string;
  required?: boolean;
  children: React.ReactNode;
}

export function FormField({ label, error, hint, required, children }: FormFieldProps) {
  const id = useId();
  return (
    <div className="flex flex-col gap-1 mb-4">
      <label htmlFor={id} className="text-sm font-medium">
        {label}
        {required && <span aria-hidden className="text-danger ml-1">*</span>}
      </label>
      {React.cloneElement(children as React.ReactElement, {
        id,
        'aria-invalid': !!error,
        'aria-describedby': error ? `${id}-error` : hint ? `${id}-hint` : undefined,
      })}
      {hint && !error && <span id={`${id}-hint`} className="text-xs text-muted">{hint}</span>}
      {error && (
        <span id={`${id}-error`} role="alert" className="text-xs text-danger">
          {error}
        </span>
      )}
    </div>
  );
}
```

## 8.7 Máscaras de entrada

Usar react-imask para CPF, telefone e datas. Não bibliotecas pesadas como react-input-mask:

| Campo      | Máscara                | Comportamento                                                                |
|------------|------------------------|------------------------------------------------------------------------------|
| CPF        | 000.000.000-00         | Aceita só números; formato visual; validador Zod com regex                   |
| Telefone   | (00) 00000-0000        | DDD + 11 dígitos brasileiros                                                 |
| Data       | dd/mm/yyyy             | Input nativo type=date OU máscara manual; converter para ISO antes do submit |
| CEP        | 00000-000              | Não usado no MVP — preservar para futuro                                     |
| Decimal kg | sem máscara, validação | Aceita vírgula OU ponto; normalizar para ponto antes                         |

| Campo | Máscara | Comportamento |
|-------|---------|---------------|
|       | Zod     | do submit     |

## 9. Performance

### 9.1 Code splitting por rota

Cada rota principal carrega seu próprio chunk. Reduz o JS inicial de ~600KB para ~150KB no first paint. Implementação com React.lazy + Suspense:

#### Code splitting por rota

```
// src/app/router.tsx
import { lazy, Suspense } from 'react';
import { createBrowserRouter } from 'react-router-dom';

// Lazy imports – Vite gera chunks separados automaticamente
const DashboardPage = lazy(() => import('@pages/dashboard/DashboardPage'));
const AnimaisListPage = lazy(() => import('@pages/animais/AnimaisListPage'));
const AnimalDetailPage = lazy(() => import('@pages/animais/AnimalDetailPage'));
const InseminacaoPage = lazy(() => import('@pages/inseminacao/InseminacaoPage'));
const AnaliseIAPage = lazy(() => import('@pages/ia/AnaliseIAPage'));
const DiarioPage = lazy(() => import('@pages/diario/DiarioPage'));
const RelatoriosPage = lazy(() => import('@pages/relatorios/RelatoriosPage'));
const PerfilPage = lazy(() => import('@pages/perfil/PerfilPage'));
const ReprodutoresPage = lazy(() => import('@pages/reprodutores/ReprodutoresPage'));

// Wrapper com fallback padrão
const LazyRoute = ({ children }: { children: React.ReactNode }) => (
  <Suspense fallback={<PageSkeleton />}>{children}</Suspense>
);

export const router = createBrowserRouter([
  { path: '/dashboard', element: <LazyRoute><DashboardPage /></LazyRoute> },
  { path: '/animais', element: <LazyRoute><AnimaisListPage /></LazyRoute> },
  // ...
]);
```

### 9.2 Lazy loading de bibliotecas pesadas

Chart.js sozinho pesa ~190KB. Carregar apenas quando o componente de gráfico é montado:

#### Lazy load de Chart.js

```
// src/components/charts/LazyReproductiveChart.tsx
const ReproductiveChart = lazy(() => import('../ReproductiveChart'));

export function LazyReproductiveChart(props: ReproductiveChartProps) {
  return (
    <Suspense fallback={<ChartSkeleton />}>
      <ReproductiveChart {...props} />
    </Suspense>
  );
}
```

```

    </Suspense>
  );
}

```

Aplicar o mesmo padrão para qualquer biblioteca >100KB que não seja crítica para o first paint.

### 9.3 Virtualização para listas grandes

Listas com mais de 100 itens visíveis (TELA-02 com 500 animais carregados) devem usar virtualização. Renderizar 500 linhas no DOM trava o scroll em mobile. Biblioteca: @tanstack/react-virtual.

| Tela                   | Quando virtualizar                        | Biblioteca              |
|------------------------|-------------------------------------------|-------------------------|
| TELA-02 (Animais)      | Sempre (rebanho pode ter centenas)        | @tanstack/react-virtual |
| TELA-03 (Inseminação)  | Histórico pode ter milhares — virtualizar | @tanstack/react-virtual |
| TELA-11 (Reprodutores) | Geralmente <50 — não virtualizar          | N/A                     |
| Diário aba Peso        | Lista de pesagens — virtualizar se >50    | @tanstack/react-virtual |
| Relatórios — prévia    | Limitada a 50 — não virtualizar           | N/A                     |
| DRAWER Alertas         | Pode ter dezenas — virtualizar se >30     | @tanstack/react-virtual |

#### Virtualização com @tanstack/react-virtual

```

// Padrão de virtualização
import { useVirtualizer } from '@tanstack/react-virtual';

function AnimaisList({ animais }: { animais: Animal[] }) {
  const parentRef = useRef<HTMLDivElement>(null);
  const rowVirtualizer = useVirtualizer({
    count: animais.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 72, // altura aproximada da linha em px
    overscan: 5, // renderiza 5 itens fora da viewport
  });

  return (
    <div ref={parentRef} style={{ height: '70vh', overflow: 'auto' }}>
      <div style={{ height: rowVirtualizer.getTotalSize(), position: 'relative' }}>
        {rowVirtualizer.getVirtualItems().map(v => (
          <div
            key={v.key}
            style={{
              position: 'absolute', top: 0, left: 0, width: '100%',
              height: v.size, transform: `translateY(${v.start}px)`
            }}
          >
            <AnimalRow animal={animais[v.index]} />
          </div>
        ))}
      </div>
    </div>
  );
}

```

## 9.4 Quando memoizar (useMemo, useCallback, React.memo)

### Regra prática

NÃO memoize por padrão. Memoize apenas quando: (1) o cálculo é caro (>5ms) E roda em re-render frequente; (2) a função/objeto é prop de componente memoizado; (3) profiler mostra problema real. Memoização excessiva polui o código e adiciona overhead de comparação que pode ser maior que o ganho.

| Caso                                                        | Memoizar?         | Por quê                                         |
|-------------------------------------------------------------|-------------------|-------------------------------------------------|
| Filtros + busca + ordenação aplicados em lista local        | SIM (useMemo)     | Cálculo recorre a cada render do componente pai |
| Lista de animais já filtrada pela API                       | NÃO               | Dados já vêm prontos; render é direto           |
| Handler onClick de botão simples                            | NÃO               | useCallback aqui é overhead sem ganho           |
| Handler passado para lista virtualizada (filhos memoizados) | SIM (useCallback) | Evita re-render dos itens                       |
| Cálculo de GMD em lista de pesagens (10 itens)              | SIM (useMemo)     | Recorre a cada interação se não memoizar        |
| Componente <KPICard> recebendo só primitivos                | NÃO               | React diff é eficiente para props simples       |

## 9.5 Imagens e assets

- Logo: SVG inline (zero requests, escalável)
- Ícones: lucide-react importados individualmente (tree-shaking automático do Vite)
- Ícones do PWA (192, 512): PNG otimizados via squoosh ou imagemin
- Sem fotos de animais no MVP — fica para v1.1

## 9.6 Métricas-alvo (Lighthouse mobile)

| Métrica                | Alvo | Crítico se |
|------------------------|------|------------|
| Performance            | ≥ 80 | < 60       |
| Acessibilidade         | ≥ 90 | < 80       |
| Best Practices         | ≥ 90 | < 80       |
| SEO                    | ≥ 80 | < 60       |
| PWA                    | ≥ 90 | < 80       |
| First Contentful Paint | < 2s | > 3s       |

| Métrica                  | Alvo  | Crítico se |
|--------------------------|-------|------------|
| Largest Contentful Paint | < 3s  | > 4s       |
| Time to Interactive      | < 4s  | > 6s       |
| Cumulative Layout Shift  | < 0.1 | > 0.25     |

## 10. Toasts e Notificações

### 10.1 Stack escolhida

Usar Radix UI Toast (já listado nas dependências). Wrapper próprio em `src/components/ui/toast.tsx` para padronizar tipos e mensagens.

### 10.2 Tipos de toast

| Tipo    | Cor                      | Duração    | Ícone (Lucide) | Quando usar                            |
|---------|--------------------------|------------|----------------|----------------------------------------|
| success | Verde (--ok-soft)        | 4 segundos | check-circle   | Operação bem-sucedida                  |
| error   | Vermelho (--danger-soft) | 6 segundos | x-circle       | Falha que não é crítica                |
| info    | Azul (--blue-soft)       | 4 segundos | info           | Notificação neutra (ex: salvo offline) |
| warning | Amarelo (--amber-soft)   | 5 segundos | alert-triangle | Alerta que precisa de atenção          |

### 10.3 API do wrapper de toast

```

Wrapper de toast
// src/components/ui/toast.tsx
import { toast as radixToast } from '@lib/toast-impl';

export function showSuccessToast(message: string) {
  radixToast({ type: 'success', message, duration: 4000 });
}

export function showErrorToast(error: unknown) {
  const { message } = getErrorMessage(error); // do error-messages.ts (seção 7.1)
  radixToast({ type: 'error', message, duration: 6000 });
}

export function showInfoToast(message: string) {
  radixToast({ type: 'info', message, duration: 4000 });
}

```

```

}

export function showWarningToast(message: string) {
  radixToast({ type: 'warning', message, duration: 5000 });
}

// Toast com ação (ex: 'Salvo offline. [Ver fila]')
export function showToastWithAction(
  type: ToastType,
  message: string,
  actionLabel: string,
  onAction: () => void,
) {
  radixToast({ type, message, action: { label: actionLabel, onAction } });
}

```

## 10.4 Deduplicação

Se a mesma mensagem é disparada várias vezes em sequência (ex: clique repetido em botão), mostrar apenas uma vez. Implementar via map de "última mensagem disparada" com TTL de 2s:

```

Deduplicação simples
// src/lib/toast-impl.ts
const recentToasts = new Map<string, number>();

export function toast(input: ToastInput) {
  const key = `${input.type}:${input.message}`;
  const lastShown = recentToasts.get(key) ?? 0;
  if (Date.now() - lastShown < 2000) return; // dedupe
  recentToasts.set(key, Date.now());
  // ...dispara o Radix Toast...
}

```

## 10.5 Posicionamento

| Viewport       | Posição                | Comportamento de empilhamento                  |
|----------------|------------------------|------------------------------------------------|
| Desktop (≥ md) | Canto superior direito | Toasts empilham verticalmente (máx 3 visíveis) |
| Mobile (< md)  | Rodapé centralizado    | Acima da bottom nav. Empilham (máx 2 visíveis) |

## 10.6 Acessibilidade dos toasts

- Container do toast com role="region" e aria-label="Notificações"
- Cada toast com role="status" (info/success) ou role="alert" (error/warning)
- aria-live="polite" para info/success, aria-live="assertive" para error/warning
- Botão de fechar com aria-label="Fechar notificação"
- Tempo de exibição ajustável (usuário com prefers-reduced-motion ganha +50% no tempo)



# 11. Acessibilidade Operacional

O Adendo UX seção 9.6 listou os princípios de acessibilidade. Esta seção detalha a implementação no código — comportamentos que devem funcionar mesmo sem testes E2E formais.

## 11.1 Focus trap em modais

Radix Dialog implementa focus trap automaticamente — Tab dentro do modal não vaza para a página atrás. Garantir que TODOS os modais (MODAL-01 a MODAL-17) usem Radix Dialog como base, não reimplemtem custom.

```
Modal com Radix Dialog
// src/components/ui/Modal.tsx – wrapper sobre Radix Dialog
import * as Dialog from '@radix-ui/react-dialog';

export function Modal({ open, onOpenChange, title, children, ...props }) {
  return (
    <Dialog.Root open={open} onOpenChange={onOpenChange}>
      <Dialog.Portal>
        <Dialog.Overlay className="fixed inset-0 bg-black/50 z-50" />
        <Dialog.Content className="...">
          {/* Radix já cuida do focus trap, Esc para fechar, aria-modal */}
          <Dialog.Title>{title}</Dialog.Title>
          <Dialog.Description
            className="sr-only">{props.description}</Dialog.Description>
            {children}
          </Dialog.Content>
        </Dialog.Portal>
      </Dialog.Root>
    );
}
```

## 11.2 Tecla Esc — comportamento padronizado

| Componente                     | Comportamento ao pressionar Esc                              |
|--------------------------------|--------------------------------------------------------------|
| Modal aberto                   | Fecha o modal (Radix Dialog faz por padrão)                  |
| Drawer lateral aberto          | Fecha o drawer                                               |
| Dropdown aberto                | Fecha o dropdown                                             |
| Combobox autocomplete          | Fecha a lista de sugestões (mas mantém o input)              |
| Form com alterações não salvas | Modal de confirmação "Descartar alterações?" antes de fechar |
| Tela cheia (sem modal)         | Nada (Esc não tem ação)                                      |

## 11.3 Skip-link "Pular para conteúdo principal"

Primeiro elemento focável da página, oculto até receber foco via Tab:

```

Skip-link
// src/components/layout/SkipLink.tsx
export function SkipLink() {
  return (
    <a
      href="#main-content"
      className="
        sr-only focus:not-sr-only
        focus:fixed focus:top-2 focus:left-2 focus:z-[100]
        focus:px-4 focus:py-2 focus:bg-primary focus:text-white
        focus:rounded-md focus:outline-none focus:ring-2
      "
    >
      Pular para o conteúdo principal
    </a>
  );
}

// Em src/components/layout/MainLayout.tsx:
<>
  <SkipLink />
  <Header />
  <main id="main-content" tabIndex={-1}>
    <Outlet />
  </main>
  <BottomNav className="md:hidden" />
</>

```

## 11.4 aria-label em ícones-only buttons

Todo botão sem texto visível precisa de aria-label descritivo:

```

aria-label em ícones
// Errado – leitor de tela diz apenas 'botão'
<button onClick={onDelete}><Trash2 /></button>

// Certo – leitor de tela diz 'Excluir animal Mimosa, botão'
<button onClick={onDelete} aria-label={`Excluir animal ${animal.nome}`}>
  <Trash2 />
</button>

// Casos comuns no AgroGen IA:
<button aria-label="Abrir notificações">                <Bell /> </button>
<button aria-label="Trocar para modo escuro">            <Moon /> </button>
<button aria-label="Abrir menu do usuário">              <User /> </button>
<button aria-label={`Ver perfil de ${nome}`}>            <Eye /> </button>
<button aria-label={`Editar ${nome}`}>                    <Pencil /> </button>
<button aria-label={`Excluir ${nome}`}>                    <Trash2 /> </button>
<button aria-label="Limpar filtros">                      <X /> </button>

```

## 11.5 Ordem do Tab

Em modais e formulários, a ordem do Tab deve seguir o fluxo visual de leitura: cima → baixo, esquerda → direita. Não usar tabIndex positivos (causam confusão); usar apenas tabIndex={0} para tornar elementos focáveis ou tabIndex={-1} para removê-los do fluxo.

## 11.6 Indicadores de foco visíveis

Todo elemento interativo precisa de outline visível ao receber foco via teclado:

```
Estilo global de foco
/* styles.css */
@layer base {
  *:focus-visible {
    outline: 2px solid var(--green-700);
    outline-offset: 2px;
    border-radius: 4px;
  }

  /* Em dark mode, ajustar contraste */
  [data-theme='dark'] *:focus-visible {
    outline-color: var(--green-300);
  }
}
```

Usar `:focus-visible` em vez de `:focus` para não exibir o anel ao clicar com mouse — só ao navegar por teclado.

## 11.7 Suporte a prefers-reduced-motion

Usuários com sensibilidade vestibular podem desativar animações no SO. Respeitar:

```
Respeitar prefers-reduced-motion
/* styles.css */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

## 11.8 Contraste mínimo

- Texto normal: contraste mínimo 4.5:1 contra o fundo (WCAG 2.1 AA)
- Texto grande ( $\geq 18$ pt ou 14pt bold): contraste mínimo 3:1
- Elementos interativos não-textuais (ícones, bordas de input): contraste mínimo 3:1
- Validar tokens light e dark com ferramenta como axe DevTools antes do deploy

## 12. Checklist de Engenharia para AUDIT.md

Adicionar este bloco ao docs/AUDIT.md que o Claude Code irá gerar. Cobre toda a parte de engenharia frontend não coberta pelos demais checklists de UX e telas.

### TanStack Query (seção 2)

- QueryClient configurado globalmente em src/app/providers.tsx
- Query keys seguem padrão ['dominio','list'|'detail',params]
- staleTime customizado por recurso (sino 60s, dashboard 5min, raças 24h)
- Toda mutation tem onSuccess com invalidateQueries explícito
- Pré-população de cache (setQueryData) em mutations de create
- Optimistic updates implementados apenas em: marcar alerta lido/resolvido, toggle ativo de reprodutor
- Matriz de invalidação por mutation respeitada (tabela 2.5)

### Axios e autenticação (seção 3)

- Instância única em src/lib/api/client.ts; nada de import axios direto fora dela
- Interceptor de request injeta Authorization e X-Fazenda-ID
- Interceptor de response trata 401 com refresh token + deduplicação via refreshPromise
- Refresh token rotacionado a cada uso
- Lembrar-me marcado: refresh\_token em localStorage. Não marcado: sessionStorage
- AbortSignal repassado em todos os endpoints de query
- useAuth com login, logout, isAuthenticated

### Offline-first (seção 4)

- Hook useOnlineStatus criado
- Banner global de status offline implementado (4 estados: online, offline, sincronizando, falha)
- Mutation queue persistida em IndexedDB via idb-keyval
- Hook useQueueSync processa queue sequencialmente quando online
- Mutations relevantes (animal, inseminação, pesagem, parto, sanitário, ocorrência, alerta) são offline-aware
- Indicadores visuais: badge PENDENTE em animais criados offline, ícone relógio em inseminações
- Limite de 50 mutations na queue com bloqueio gracioso ao atingir
- Mutations não permitidas offline (login, predição IA, geração PDF, exclusões) bloqueadas com mensagem clara

## Filtros (seção 5)

- Chips/pills exibem contagem ao lado de cada opção
- Range numérico usa dois inputs (não slider) para peso e idade
- Date range com atalhos: Hoje, 7 dias, Este mês, Último trimestre, Este ano, Tudo
- useDebounce com 300ms para busca textual; 500ms para validação assíncrona
- Filtros persistidos em sessionStorage durante a sessão
- Botão "Limpar filtros" visível apenas quando há filtros ativos
- DRAWER-02 e DRAWER-03 implementados com todos os filtros listados

## Loading e skeletons (seção 6)

- Componentes de skeleton criados: TableSkeleton, CardListSkeleton, KPICardSkeleton, ChartSkeleton, AnimalProfileSkeleton, FormSkeleton, DiarioTabSkeleton
- Durante refetch, dados antigos ficam visíveis com indicador sutil (placeholderData: keepPreviousData)
- Botões de submit com spinner inline + disabled durante mutation
- Loading da Predição IA com mensagens rotativas
- Geração de PDF com banner "continue navegando" após 3s

## Erros (seção 7)

- src/lib/api/error-messages.ts criado e usado em todos os catch
- ErrorBoundary global envolve a aplicação
- Rota catch-all (\*) com NotFoundPage amigável
- Tela de erro inline com botão "Tentar de novo" em queries que falham
- Mensagens sempre em pt-BR, nunca expondo códigos técnicos

## Formulários (seção 8)

- React Hook Form + Zod usado em TODOS os formulários (não usar useState manual)
- Schemas em src/lib/schemas/ — um arquivo por entidade
- Schemas reaproveitados entre create e edit (com extend/refine)
- Validação onBlur por padrão, revalidação onChange após primeiro erro
- Validação assíncrona (brinco único) com debounce 500ms
- FormField padronizado em src/components/ui/FormField.tsx
- Máscaras com react-imask em CPF e telefone
- Tipos TypeScript derivados via z.infer dos schemas

## Performance (seção 9)

- Code splitting com React.lazy em todas as rotas principais
- Chart.js carregado lazy via LazyReproductiveChart e similares
- Virtualização com @tanstack/react-virtual em listas com 100+ itens
- Memoização aplicada com critério (não excessivamente)
- Lighthouse mobile: Performance  $\geq 80$ , A11y  $\geq 90$ , PWA  $\geq 90$

## Toasts (seção 10)

- Wrapper em src/components/ui/toast.tsx com showSuccess/Error/Info/WarningToast
- 4 tipos visuais (success/error/info/warning) com durações distintas
- Deduplicação de mensagens idênticas em janela de 2s
- Posicionamento mobile: rodapé acima da bottom nav. Desktop: canto superior direito
- aria-live="polite" para info/success, "assertive" para error/warning

## Acessibilidade (seção 11)

- Todos os modais usam Radix Dialog (focus trap nativo)
- Esc fecha modais, drawers, dropdowns e comboboxes
- Skip-link "Pular para conteúdo principal" no topo
- aria-label descritivo em todos os ícone-only buttons
- main com id="main-content" e tabIndex={-1}
- :focus-visible com outline 2px na cor primária
- @media (prefers-reduced-motion) respeitado
- Contraste mínimo 4.5:1 validado em light e dark

### Posição no plano

Os itens deste documento são transversais — não cabem em um único commit. Aplicar à medida que os componentes vão sendo construídos: a estratégia TanStack é criada no commit 2 (setup), a camada Axios no commit 3 (api layer), offline-first como módulo isolado no commit 5, formulários conforme os modais são implementados (commits 6-9), performance e a11y em revisão final (commit 10). Manter o AUDIT.md atualizado a cada commit.